Simulation framework for Table 1

J. Dedecker, M.-L. Taupin, V. Runge and A.-S. Tocquet

2026-07-16

Contents

1 Objective

This document implements the continuous, heteroscedastic Table 1 data-generating process with a faster simulator for the correlated Gaussian blocks.

The improvement replaces dense covariance-matrix construction and Cholesky factorization by the exact stationary Gaussian AR(1) recursion

$$X_j = \rho X_{j-1} + \sqrt{1-\rho^2}\,\varepsilon_j, \qquad \varepsilon_j \overset{i.i.d.}{\sim} \mathcal{N}(0, \sigma^2),$$

which gives

$$\mathrm{Cov}(X_j, X_k) = \sigma^2 \rho^{|j-k|}.$$

Thus the simulated covariance is the same Toeplitz covariance as in Table 1, but the large 1982-variable noise block is generated much faster.

2 Packages

```
knitr::opts_chunk$set(echo = TRUE, eval = TRUE, cache.path = "table1_cache/")
library(parallel)
```

3 Global parameters

```
# Table 1 dimensions
p <- 2000L
N <- 500L

#set.seed(20260701)
base_seed <- 20260701L
replication_seeds <- base_seed + seq_len(N)

n_values <- c(500L, 1000L, 2000L, 2500L)

# Variable groups
n_active <- 10L
n_proxy_and_noise <- 8L
n_toeplitz_noise <- p - n_active - n_proxy_and_noise  # 1982

# Toeplitz correlations
rho_active <- 0.57
rho_toeplitz_noise <- 0.70

# Regression coefficients, including the intercept
theta_star <- c(
  -1,
   3.619350,
  -3.274923,
```

```
   2.963273,
  -2.681280,
   2, 4, 6, 3, 2, 4
)

stopifnot(length(theta_star) == n_active + 1L)
stopifnot(n_toeplitz_noise == 1982L)
```

4 Fast Toeplitz-Gaussian generator

```
simulate_toeplitz_gaussian_fast <- function(n, d, rho, sigma = 1)
{
  stopifnot(d >= 1L, abs(rho) < 1, sigma > 0)

  X <- matrix(0, nrow = n, ncol = d)

  # A stationary AR(1) process starts with marginal variance sigma^2.
  X[, 1L] <- rnorm(n, mean = 0, sd = sigma)

  if (d > 1L) {
    innovation_sd <- sigma * sqrt(1 - rho^2)

    # Draw all innovations at once to reduce random-number-generation overhead.
    innovations <- matrix(
      rnorm(n * (d - 1L), mean = 0, sd = innovation_sd),
      nrow = n,
      ncol = d - 1L
    )

    # Each row is a stationary Gaussian AR(1) sequence.
    for (j in 2:d) {
      X[, j] <- rho * X[, j - 1L] + innovations[, j - 1L]
    }
  }

  X
}
```

5 One Table 1 data set

This function returns only X and Y by default. It does not create column names or duplicate metadata at every replication, reducing memory allocation in Monte Carlo experiments.

```
simulate_table1_dataset_fast <- function(
    n,
    p = 2000,
    rho_active = 0.57,
    rho_toeplitz_noise = 0.70,
    theta_star = c(-1, 3.619350, -3.274923, 2.963273, -2.681280, 2, 4, 6, 3,2, 4),
```

```
    add_names = FALSE
) {
  # ------------------------------------------------------------------
  # Group A: X1,...,X10
  # ------------------------------------------------------------------

  X1_to_X5 <- simulate_toeplitz_gaussian_fast(
    n = n,
    d = 5L,
    rho = rho_active,
    sigma = 1
  )

  X6 <- rbinom(n, size = 1L, prob = 0.35)
  X7 <- rchisq(n, df = 2)
  X8 <- X1_to_X5[, 1L] * rpois(n, lambda = 2)
  X9 <- X1_to_X5[, 2L] * rnorm(n, mean = 1, sd = 1)
  X10 <- rt(n, df = 14)

  X_active <- cbind(X1_to_X5, X6, X7, X8, X9, X10)

  # ------------------------------------------------------------------
  # Response Y
  # ------------------------------------------------------------------

  linear_predictor <- drop(theta_star[1L] + X_active %*% theta_star[-1L])
  epsilon <- X_active[, 5L] * (rchisq(n, df = 3) - 3)
  Y <- abs(linear_predictor)^0.8 + epsilon / 3

  # ------------------------------------------------------------------
  # Groups B and C: X11,...,X18
  # ------------------------------------------------------------------

  Z3 <- rnorm(n, mean = 0, sd = 0.40)
  Z4 <- rnorm(n, mean = 0, sd = 0.02)
  Z5 <- rnorm(n, mean = 0, sd = 0.35)
  Z6 <- rnorm(n, mean = 0, sd = 0.55)

  X_proxy_and_noise <- cbind(
    X_active[, 1L] + Z3,  # X11
    X_active[, 3L] + Z4,  # X12
    X_active[, 4L] + Z5,  # X13
    X_active[, 5L] + Z6,  # X14
    Z3, Z4, Z5, Z6        # X15,...,X18
  )

  # ------------------------------------------------------------------
  # Group D: X19,...,X2000
  # ------------------------------------------------------------------

  X19_to_X2000 <- simulate_toeplitz_gaussian_fast(
    n = n,
    d = p - 18L,
```

```
    rho = rho_toeplitz_noise,
    sigma = 1
  )

  X <- cbind(X_active, X_proxy_and_noise, X19_to_X2000)

  stopifnot(ncol(X) == p)
  stopifnot(length(Y) == n)

  if (add_names) {
    colnames(X) <- paste0("X", seq_len(p))
  }

  list(X = X, Y = Y)
}
```

6 Shared metadata

Keep this object once, outside Monte Carlo replications.

```
table1_metadata <- list(
  p = p,
  N = N,
  n_values = n_values,
  theta_star = theta_star,
  rho_active = rho_active,
  rho_toeplitz_noise = rho_toeplitz_noise,
  active_indices = 1:10,
  proxy_indices = 11:14,
  null_noise_indices = 15:18,
  toeplitz_noise_indices = 19:p,
  table1_positive_indices = 1:14,
  table1_null_indices = 15:p
)
```

7 Example data set

```
example_dataset <- simulate_table1_dataset_fast(
  n = 500L,
    add_names = FALSE
)

dim(example_dataset$X)
```

```
## [1]  500 2000
```

```
head(example_dataset$X[, 1:12])
```

```
##                                                                 X6        X7
## [1,] -0.2984047  1.4172211  0.47541354 -0.03451679  0.1165706  0 0.6578737
## [2,] -0.5486147  0.7423427  1.26406714 -0.34688027 -0.1742070  0 1.8492509
## [3,]  0.8237802  1.2723323 -0.04987161  0.44036282  1.0097871  1 4.7515926
## [4,]  0.9182469 -0.4323958 -1.32387855 -0.94649269 -1.3365333  0 0.2683141
## [5,] -0.4672442 -1.5016816  0.13814466 -0.75498784  2.2484627  1 2.6480928
## [6,]  0.5408649  0.6780089  0.62842358 -0.75566377  0.2716344  0 2.1329315
##              X8         X9        X10
## [1,] -0.8952141  0.4263195 -1.8757867 -0.6556565  0.42341882
## [2,]  0.0000000  1.3397921  0.6838330 -0.2908787  1.27164984
## [3,]  1.6475604  2.2090448  2.3125966  0.5957615 -0.05944325
## [4,]  0.0000000 -1.5144840 -1.1793855  0.9078135 -1.34937825
## [5,] -1.8689768 -2.1979885  0.5614187 -0.8553750  0.11799944
## [6,]  1.6225948  0.3770722 -1.5695078  0.1497297  0.60472179
```

```
head(example_dataset$Y)
```

```
## [1]  6.472785  9.082721 22.114452  5.753229 12.656632  8.765436
```

```
# Empirical correlation of X1,...,X5, to inspect the rho = 0.57 block:
round(cor(example_dataset$X[, 1:5]), 3)
```

```
##
##  1.000 0.633 0.386 0.213 0.106
##  0.633 1.000 0.598 0.336 0.123
##  0.386 0.598 1.000 0.567 0.297
##  0.213 0.336 0.567 1.000 0.545
##  0.106 0.123 0.297 0.545 1.000
```

```
# Empirical correlation of X150,...,X155, to inspect the rho = 0.7 block:
round(cor(example_dataset$X[, 150:155]), 3)
```

```
##
##  1.000 0.711 0.446 0.340 0.228 0.126
##  0.711 1.000 0.652 0.480 0.353 0.256
##  0.446 0.652 1.000 0.722 0.534 0.376
##  0.340 0.480 0.722 1.000 0.700 0.447
##  0.228 0.353 0.534 0.700 1.000 0.670
##  0.126 0.256 0.376 0.447 0.670 1.000
```

8 Screening function from the `mfscreen` package

The screening routine is provided by the GitHub package `vrunge/mfscreen`. The following chunk installs `remotes` only when needed, installs `mfscreen` only when it is not already available, and then loads the package.

```
if (!requireNamespace("remotes", quietly = TRUE)) {
  install.packages("remotes")
}
```

```r
if (!requireNamespace("mfscreen", quietly = TRUE)) {
  remotes::install_github("vrunge/mfscreen")
}

library(mfscreen)

# Confirm that the required function is available.
stopifnot(exists("screening_test_matrix", where = asNamespace("mfscreen")))
```

9 One replication: data generation and screening

This function generates one Table 1 data set, immediately applies the Rcpp screening function, and returns only the quantities needed for Table 1. The $n \times p$ design matrix is not retained after the function returns.

```r
run_one_table1_replication <- function(n, q = 0.10) {
  dat <- simulate_table1_dataset_fast(
    n = n,
    p = p,
    rho_active = rho_active,
    rho_toeplitz_noise = rho_toeplitz_noise,
    theta_star = theta_star,
    add_names = FALSE
  )

  test_result <- mfscreen::screening_test_matrix(
    X = dat$X,
    y = as.numeric(dat$Y),
    q = q
  )

  selected <- as.logical(test_result$selected)

  # Table 1: X1,...,X14 are treated as positive; X15,...,X2000 are null.
  selected_first18 <- as.integer(selected[1:18])

  output <- list(
    selected_first18 = selected_first18,
    selected_positive = sum(selected[1:14]),
    selected_null = sum(selected[15:p]),
    selected_total = sum(selected),
    threshold = test_result$threshold
  )

  output
}
```

10 Example: one replication

```r
one_result <- run_one_table1_replication(n = 500L, q = 0.10)

one_replication_summary <- data.frame(
  n = 500L,
  p = p,
  q = 0.10,
  threshold = one_result$threshold,
  selected_among_X1_to_X14 = one_result$selected_positive,
  selected_among_X15_to_X2000 = one_result$selected_null,
  selected_model_size = one_result$selected_total,
  TPR_contribution = one_result$selected_positive / 14,
  FPR_contribution = one_result$selected_null / (p - 14)
)

knitr::kable(
  one_replication_summary,
  digits = 4,
  caption = "One replication: quantities that are averaged for Table 1"
)
```

Table 1: One replication: quantities that are averaged for Table 1

n	p	q	threshold	selected_among_X1_to_X14	selected_among_X15_to_X2000	selected_model_size	TPR_contribution	FPR_contribution
500	2000	0.1	0.3696	13	182	195	0.9286	0.0916

```r
knitr::kable(
  data.frame(
    variable = paste0("X", 1:18),
    selected = one_result$selected_first18
  ),
  caption = "One replication: selection indicators for X1 to X18"
)
```

Table 2: One replication: selection indicators for X1 to X18

variable	selected
X1	1
X2	1
X3	1
X4	1
X5	1
X6	0
X7	1
X8	1
X9	1
X10	1
X11	1
X12	1
X13	1
X14	1

variable	selected
X15	0
X16	0
X17	0
X18	0

11 Monte Carlo simulation for one Table 1 column

The function below runs N replications for a fixed sample size and returns only compact outputs. It does not save any X or Y matrices.

```
run_table1_column <- function(n, N = N, q = 0.10,
                              mc.cores = 1L,
                              base_seed = 20260701L)
{

  replication_seeds <- base_seed + seq_len(N)

  one_draw <- function(i) {
    set.seed(replication_seeds[i])
    run_one_table1_replication(n = n, q = q)
  }

  if (.Platform$OS.type == "windows" || mc.cores <= 1L) {
    repetitions <- lapply(seq_len(N), one_draw)
  } else {
    repetitions <- parallel::mclapply(
      X = seq_len(N),
      FUN = one_draw,
      mc.cores = mc.cores,
      mc.set.seed = FALSE,
      mc.preschedule = FALSE
    )
  }

  selection_first18 <- do.call(
    rbind,
    lapply(repetitions, `[[`, "selected_first18")
  )

  selected_positive <- vapply(
    repetitions,
    `[[`,
    numeric(1),
    "selected_positive"
  )

  selected_null <- vapply(
    repetitions,
    `[[`,
    numeric(1),
```

```
    "selected_null"
  )

  selected_total <- vapply(
    repetitions,
    `[[`,
    numeric(1),
    "selected_total"
  )

  list(
    n = n,
    N = N,
    q = q,
    threshold = repetitions[[1]]$threshold,
    selection_frequency_first18 = colMeans(selection_first18),
    TPR = mean(selected_positive / 14),
    FPR = mean(selected_null / (p - 14)),
    mean_selected_model_size = mean(selected_total)
  )
}
```

12 Generate Table 1

This chunk is evaluated when the document is knitted. It runs the simulation for $n = 500, 1000, 2000, 2500$, each with $N = 500$ Monte Carlo replications, and then creates the generated Table 1 in the PDF.

For a quick development check, temporarily replace `N <- 500L` in the parameter section by a smaller value such as `N <- 5L`.

```
# On machines with limited RAM, start with 4 to 8 cores.
n_cores <- min(8L, parallel::detectCores(logical = FALSE))

table1_runs <- lapply(
  n_values,
  run_table1_column,
  N = N,
  q = 0.10,
  mc.cores = n_cores
)

names(table1_runs) <- paste0("n=", n_values)
```

13 Generated Table 1

The generated Table 1 reports one row for each sample size and the three Monte Carlo quantities reported in the paper: TPR, FPR, and the mean selected model size.

```
table1_summary <- data.frame(
  n = n_values,
```

Table 3: Generated Table 1: Monte Carlo screening results (N = 500, p = 2000, q = 0.10)

	n	TPR	FPR	Mean_selected_model_size
n=500	500	0.816	0.103	216.722
n=1000	1000	0.899	0.102	214.478
n=2000	2000	0.961	0.101	213.930
n=2500	2500	0.977	0.101	214.250

```
  TPR = vapply(table1_runs, `[[`, numeric(1), "TPR"),
  FPR = vapply(table1_runs, `[[`, numeric(1), "FPR"),
  Mean_selected_model_size = vapply(
    table1_runs,
    `[[`,
    numeric(1),
    "mean_selected_model_size"
  )
)

knitr::kable(
  table1_summary,
  format = "latex",
  booktabs = TRUE,
  digits = 3,
  align = c("c", "c", "c", "c"),
  caption = paste0(
    "Generated Table 1: Monte Carlo screening results ",
    "(N = ", N, ", p = ", p, ", q = 0.10)"
  )
)
```

14 Selection frequencies for X1 to X18

This table contains the Monte Carlo selection frequencies for the first 18 covariates. The Table 1 TPR is the mean of the first 14 rows; the FPR is computed over all $X_{15}, \ldots, X_{2000}$, not only $X_{15}, \ldots, X_{18}$.

```
selection_first18_table <- data.frame(
  variable = paste0("X", 1:18),
  do.call(
    cbind,
    lapply(
      table1_runs,
      function(x) x$selection_frequency_first18
    )
  ),
  check.names = FALSE
)

colnames(selection_first18_table)[-1] <- names(table1_runs)
```

Table 4: Monte Carlo selection frequencies for X1 to X18

variable	n=500	n=1000	n=2000	n=2500
X1	1.000	1.000	1.000	1.000
X2	0.998	1.000	1.000	1.000
X3	0.994	1.000	1.000	1.000
X4	0.372	0.524	0.766	0.856
X5	0.622	0.882	0.980	1.000
X6	0.598	0.860	0.988	0.996
X7	1.000	1.000	1.000	1.000
X8	0.994	1.000	1.000	1.000
X9	0.998	1.000	1.000	1.000
X10	0.990	1.000	1.000	1.000
X11	1.000	1.000	1.000	1.000
X12	0.992	1.000	1.000	1.000
X13	0.350	0.526	0.752	0.840
X14	0.518	0.800	0.968	0.990
X15	0.114	0.090	0.102	0.078
X16	0.108	0.086	0.114	0.118
X17	0.106	0.088	0.108	0.118
X18	0.104	0.108	0.114	0.118

```
selection_first18_table[, -1] <- lapply(
  selection_first18_table[, -1, drop = FALSE],
  round,
  digits = 3
)

knitr::kable(
  selection_first18_table,
  format = "latex",
  booktabs = TRUE,
  caption = "Monte Carlo selection frequencies for X1 to X18"
)
```

15 Save compact results

```
saveRDS(
  object = list(
    metadata = table1_metadata,
    table1_summary = table1_summary,
    table1_runs = table1_runs
  ),
  file = "table1_rcpp_screening_results.rds"
)

write.csv(
```

```
  table1_summary,
  file = "table1_rcpp_screening_summary.csv",
  row.names = FALSE
)
```